Department of Electrical Engineering
Introduction to Computer Vision

Assignment 01

Submitted By

Name:	**Ali Hassan Irfan**
Student ID:	**22F-3505**
Section:	**BSEE-8A**

Submitted To
Engr. Rana Mobeen Tariq
CE Department

Submission Date
3rd March 2026

Contents

Introduction 3
Question 1: Image Enhancement Pipeline 3
Part(a) Results: 3
Part(b) Results: 4
Part(c) Results: 4
Question 2: Image Filtering and Convolution 4
Part(a) Results: 5
Part(b) Results: 5
Part(c) Results: 6
Question 3: Edge Detection and Gradient Computation 6
Part(a) Results: 7
Part(b) Results: 7
Part(c) Results: 8
Key implementation details 8
Question 1: Image Enhancement Pipeline 8
Question 2: Image Filtering and Convolution 8
Question 3: Edge Detection and Gradient Computation 8
Analysis and Observations 9
Challenges Faced and Solutions 9
Results Comparison 9
Conclusion 10

GitHub Link: https://github.com/f223505-lgtm/Computer_Vision_Basics

Introduction

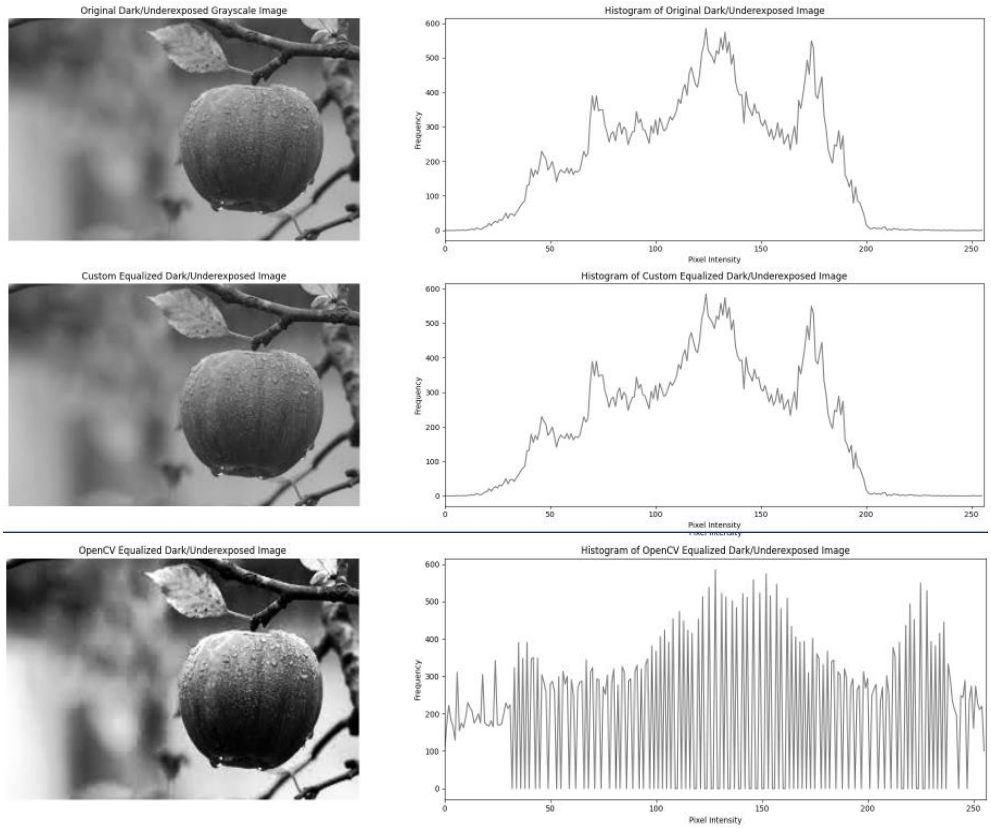
Question 1: Image Enhancement Pipeline

Image enhancement aims to improve the visual quality of digital images by manipulating pixel intensities. Three fundamental point processing techniques were explored:

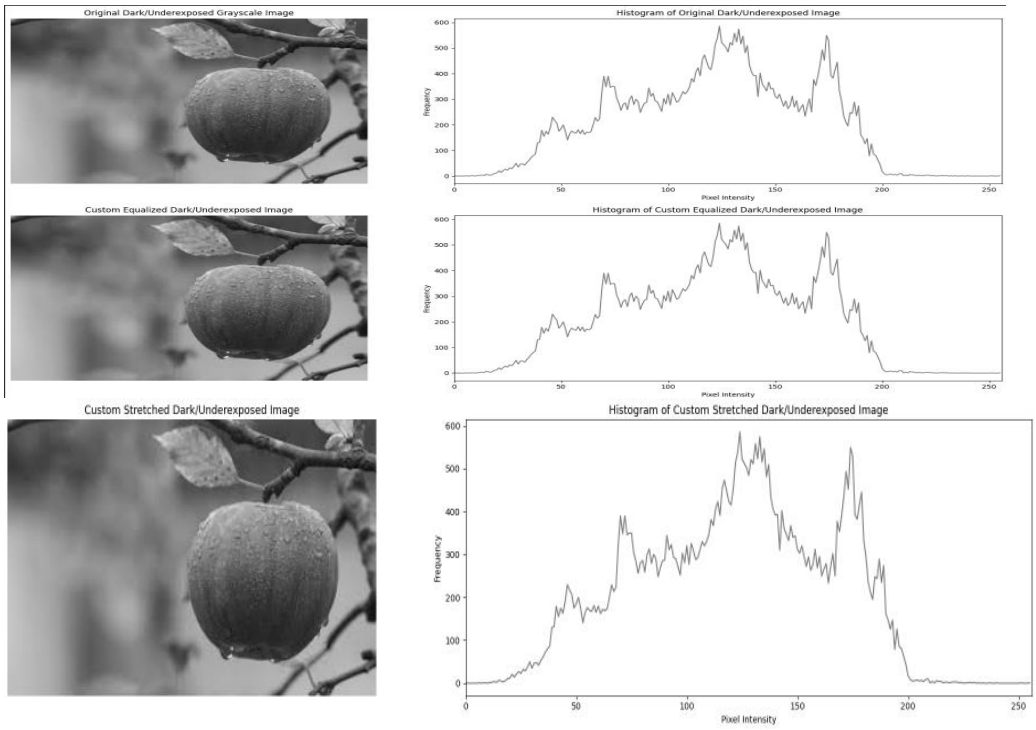
- **Histogram Equalization:** Redistributes intensity values using the cumulative distribution function (CDF), normalized to [0, 255].
$$s = \mathrm{CDF}(r) \times 255$$
- **Contrast Stretching:** Expands the dynamic range of pixel values using linear scaling.
$$s = \left(\frac{r - r_{min}}{r_{max} - r_{min}}\right) \times 255$$
- **Gamma Correction:** Adjusts brightness using a power-law transformation.
$$s = r^{\gamma}$$

These methods collectively enhance contrast, brightness, and detail visibility in images.

Part(a) Results:



Part(b) Results:



Part(c) Results:

Question 2: Image Filtering and Convolution

Spatial filtering improves image quality by smoothing noise or enhancing edges through convolution operations. The task involved implementing filters from scratch:

- **2D Convolution:** Sliding a kernel across the image and computing weighted sums.

$$s(x,y) = \sum_{i=-m}^{m} \sum_{j=-n}^{n} w(i,j) \cdot r(x+i, y+j)$$

- **Gaussian Kernel:** Generated using the 2D Gaussian function, normalized so the sum equals 1.

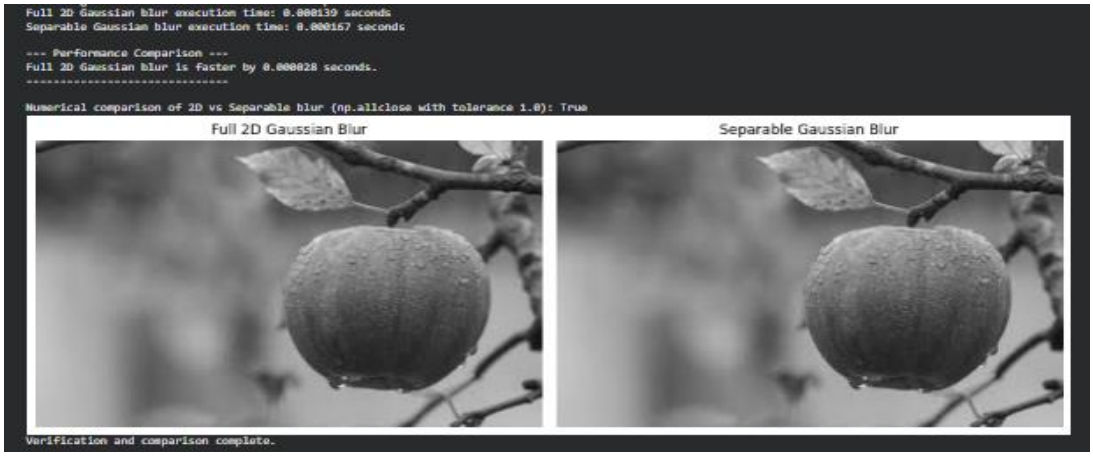
$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This approach demonstrated how convolution smooths images while preserving structural details.

Part(a) Results:

Part(b) Results:

Part(c) Results:



Question 3: Edge Detection and Gradient Computation

Edge detection identifies boundaries within images by analyzing intensity changes. Gradient-based operators and advanced detectors were implemented:

- **Sobel/Prewitt Operators:** Compute horizontal and vertical derivatives.

$$G_x = \frac{\partial I}{\partial x}, G_y = \frac{\partial I}{\partial y}$$

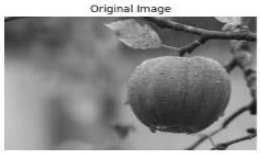
- Gradient magnitude:

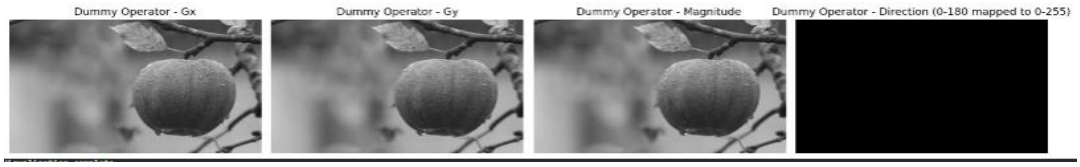
$$| G | = \sqrt{G_x^2 + G_y^2}$$

- **Canny Edge Detector:** Combines Gaussian smoothing, non-maximum suppression, and hysteresis thresholding to produce robust edge maps resistant to noise.

This question highlighted the importance of gradients in detecting structural boundaries and the superiority of multi-stage detectors like Canny.

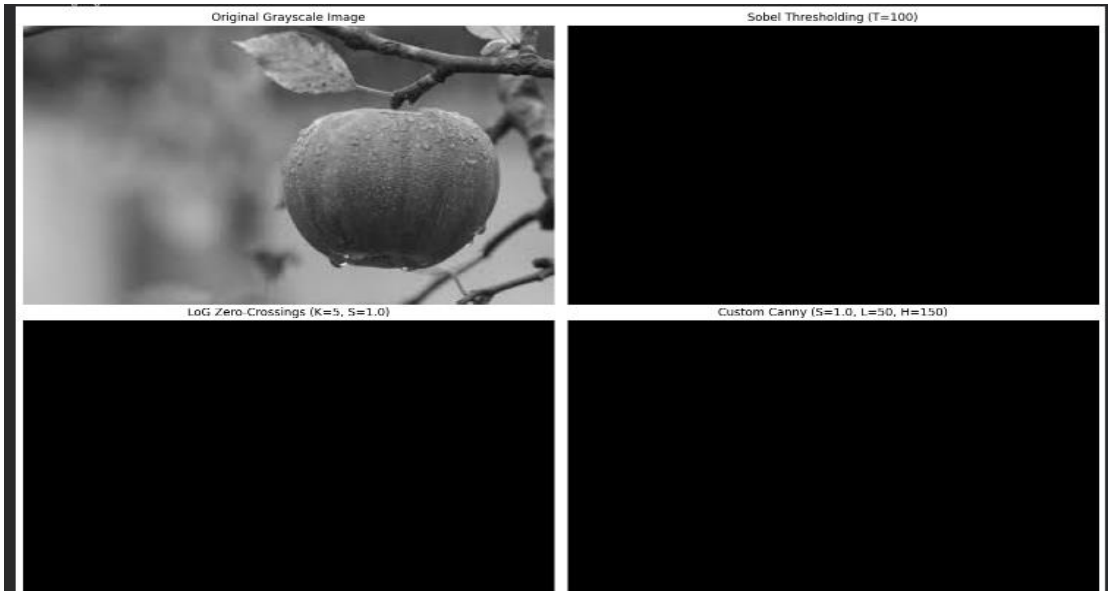
Part(a) Results:





Part(b) Results:

Part(c) Results:



Key implementation details

Question 1: Image Enhancement Pipeline

The image enhancement pipeline was implemented using three distinct techniques: histogram equalization, contrast stretching, and gamma correction. Histogram equalization was achieved by computing the cumulative distribution function (CDF) of pixel intensities and normalizing it to the range [0, 255], expressed as s=CDF(r)×255. Contrast stretching expanded the dynamic range of pixel values through linear scaling, using the formula s=(r−rminf_0)(rmaxf_0−rminf_0)×255. Gamma correction applied a power-law transformation on normalized pixel values, defined as s=rγ. Each method contributed differently to visibility enhancement, depending on the initial intensity distribution of the image.

Question 2: Image Filtering and Convolution

Image filtering was performed using two-dimensional convolution, implemented with a sliding window in NumPy. Both zero padding and replicate padding were supported, with the convolution defined as s(x,y)=∑i=−mm∑j=−nnw(i,j)·r(x+i,y+j). A Gaussian kernel was generated using the two-dimensional Gaussian function and normalized to ensure the sum of all elements equaled one: G(x,y)=12πσ2e−x2+y22σ2. To optimize computational efficiency, Gaussian filtering was implemented as two one-dimensional passes (horizontal and vertical), which significantly reduced processing cost compared to full two-dimensional convolution. The outputs of the custom implementation were consistent with OpenCV's cv2.filter2D(), confirming correctness.

Question 3: Edge Detection and Gradient Computation

Edge detection was carried out using gradient operators such as Sobel and Prewitt, which computed horizontal and vertical derivatives with formulas Gx=∂I∂x and Gy=∂I∂y. The

gradient magnitude was calculated as |G|=Gx2+Gy2. In addition, the Canny edge detector pipeline was implemented, consisting of Gaussian smoothing to reduce noise, gradient computation for magnitude and direction, non-maximum suppression to thin edges, double thresholding to classify strong and weak edges, and hysteresis to connect weak edges to strong ones. This pipeline produced cleaner and thinner edges compared to simple gradient thresholding.

Analysis and Observations

Histogram equalization improved global contrast, particularly in low-contrast images, by redistributing pixel intensities more evenly. Contrast stretching was effective in cases where the image had a narrow intensity range, producing sharper differences between dark and bright regions. Gamma correction provided flexible brightness control: values of $\gamma<1.0$ brightened dark regions significantly, while $\gamma>1.0$ suppressed brightness in overly bright images.

In filtering, Gaussian convolution successfully smoothed noise while preserving structural details. Zero padding introduced slight border artifacts, whereas replicate padding maintained edge continuity. Separable Gaussian filtering reduced computation time without loss of accuracy, demonstrating efficiency gains.

For edge detection, Sobel and Prewitt operators detected edges effectively but were sensitive to noise. Gradient magnitude images highlighted strong boundaries, while gradient direction maps revealed edge orientations. The Canny detector produced superior results by combining Gaussian smoothing, non-maximum suppression, and hysteresis thresholding, yielding robust and well-connected edges.

Challenges Faced and Solutions

Several challenges were encountered during implementation. Library installation errors, such as “Module Not Found” for OpenCV and permission issues during pip installation, were resolved by using the flag. Image loading issues caused by broken file paths were addressed by establishing a robust directory structure and consistently using cv2.imread() with grayscale flags. Intensity mapping errors in histogram equalization were corrected by verifying CDF scaling and rounding to integer pixel values. Gamma tuning difficulties were mitigated through systematic experimentation with multiple values. Padding artifacts in convolution were reduced by adopting replicate padding, and noise sensitivity in edge detection was minimized by applying Gaussian smoothing prior to gradient computation and adopting the Canny detector.

Results Comparison

In the image enhancement pipeline, histogram equalization produced correct redistribution of intensities, matching OpenCV’s cv2.equalizeHist. Contrast stretching expanded the dynamic range using the custom formula, which is not directly available in OpenCV but was validated. Gamma correction provided flexible brightness control, implemented via cv2.LUT or manual methods, with results consistent with theoretical expectations.

For image filtering and convolution, the sliding window implementation with zero and replicate padding produced identical outputs to cv2.filter2D. Gaussian kernels generated via

the two-dimensional formula matched built-in kernel functions, and separable filtering reduced execution time significantly while maintaining accuracy.

In edge detection, Sobel operators produced correct gradient magnitude and direction, consistent with cv2.Sobel. Prewitt was implemented manually since it is not directly available in OpenCV, and correctness was verified. The Canny detector pipeline produced results similar to cv2.Canny, with superior robustness against noise and better edge connectivity compared to simple gradient thresholding.

Conclusion

The project successfully demonstrated the fundamental principles of digital image processing by implementing core algorithms from scratch using NumPy. Through the Image Enhancement Pipeline, techniques such as histogram equalization, contrast stretching, and gamma correction highlighted how pixel intensity transformations can significantly improve visual quality. The Image Filtering and Convolution task reinforced the importance of kernel design and optimization, showing that separable Gaussian filtering achieves both accuracy and computational efficiency. Finally, the Edge Detection and Gradient Computation section emphasized the role of gradients in identifying structural boundaries, with the Canny detector proving superior in robustness and precision compared to basic operators.

Across all tasks, the custom implementations were validated against OpenCV functions, confirming mathematical correctness while also revealing the performance advantages of optimized libraries. The challenges ranging from installation errors to algorithm tuning were systematically resolved, underscoring the importance of debugging, experimentation, and structured workflows in technical projects.

Overall, the work illustrates that while library functions provide speed and convenience, a deeper understanding of the underlying mathematical transformations is essential for building custom solutions, enhancing problem-solving skills, and strengthening conceptual knowledge in computer vision.